



UNIT TESTING

INTRODUCTION

- Unit tests are segments of code written to test other pieces of code, typically a single function or method within a class or an entire class, that we refer to as a unit.
- they help to ensure that code works as intended and catch bugs early on.
- In Python, this is done with the unittest framework, which is built into the standard library.
- The unittest module is widely used because:
 - No setup needed:** comes pre-installed with Python.
 - Structured:** supports test cases, fixtures, assertions, and test suites.
 - Automated:** can discover and run tests across the project.
 - Stable and versatile:** reliable for projects of any size, with easy integration into other tools.

Why unittest is important?

- **Confidence in Change:** When you refactor or add new features, tests act as a safety net, immediately alerting you if you broke existing functionality.
- **Early Bug Detection:** Bugs are cheaper and easier to fix when they are found in a small, isolated unit rather than after integrating many complex parts.
- **Design Quality:** Code that is easy to unit test is typically well-designed, modular, and has low coupling, which are hallmarks of good software architecture.

THE UNITTEST MODULE

- unittest is a **built-in Python module** used for **testing code automatically**.
- It allows you to write test cases to check whether your functions, classes, or whole program works correctly.

Why Use unittest?

- ✍ Detects errors early
- ✍ Ensures code reliability
- ✍ Helps during debugging and future development
- ✍ Required in real-world applications

Feature	Description
TestCase class	Base class to create tests
Assertions	Methods to compare expected vs actual output
Test Suite	Group multiple tests
Test Runner	Executes tests and shows results

ASSERT

- Built-in statement used to if a given condition is true or not.
- If the condition is true, nothing happens, but if it's not true, an error is raised.

Method	Description
<code>.assertEqual(a, b)</code>	Checks if a is equal to b, similar to the expression <code>a == b</code> .
<code>.assertTrue(x)</code>	Asserts that the boolean value of x is True, equivalent to <code>bool(x) is True</code> .
<code>.assertIsInstance(a, b)</code>	Asserts that a is an instance of class b, similar to the expression <code>isinstance(a, b)</code> .
<code>.assertIsNone(x)</code>	Ensures that x is None, similar to the expression <code>x is None</code> .
<code>.assertFalse(x)</code>	Asserts that the boolean value of x is False, similar to <code>bool(x) is False</code> .
<code>.assertIs(a, b)</code>	Verifies if a is identical to b, akin to the expression <code>a is b</code> .
<code>.assertIn(a, b)</code>	Checks if a is a member of b, akin to the expression <code>a in b</code> .



END..